第3回：Webアプリへの組み込み

ブラウザからFastAPIを経由してLLMを呼ぶ

今日のゴールと流れ

質問がどこで処理されるかを追いかける。

時間	内容
10分	前回のAPI呼び出しの復習
20分	画面とFastAPIの役割
20分	POST・JSON・入力検証
30分	fetchの実装と動作確認
10分	エラー確認・提出

通信の2段階

1. ブラウザが自分のFastAPIに質問を送る
2. FastAPIが外部のLLM APIに質問を送る
3. LLMの結果をFastAPIが受け取る
4. FastAPIがJSONを返し、ブラウザが表示する

ブラウザへ送るのは回答です。APIキーを送る必要はありません。

質問が通るファイル

順番	実行場所	ファイルと仕事
1	ブラウザ	app.js が質問をJSONにする
2	サーバー	main.py がJSONを受け取り検証する
3	サーバー	llm.py がLLM APIを呼ぶ
4	サーバー	main.py が結果の辞書を返す
5	ブラウザ	app.js が回答を表示する

APIキーを使うのは、サーバー側のLLM呼び出しです。

既存コードと今回の追加

前回から使う	今回加える
`settings.py`	`main.py` ：自作API
`llm.py` の `ask_plain()`	`static/index.html` ：入力画面
LLM APIの呼び出し	`static/app.js` ：送信と表示
入出力の辞書	`static/style.css` ：見た目

`session03` のコードには、検索やDBの機能はまだありません。

質問のJSON

ブラウザからFastAPIへ送る例です。

```
{
  "question": "FastAPIとは？",
  "mode": "plain",
  "search_text": "",
  "top_k": 3
}
```

`search_text` と `top_k` は、後の検索用です。今回は使いません。

同じ質問の3つの形

場所	表現
JavaScriptのオブジェクト	`{question: 'FastAPIとは？'}`
HTTPで送るJSON文字列	`{"question":"FastAPIとは？"}`
Pythonの受信後	`request.question` が文字列を持つ

`JSON.stringify()` は送信用の文字列へ変換します。
サーバーはJSONを読み取り、入力モデルで検証します。
質問そのものの意味は、変換の前後で変えません。

Pydanticによる入力の形

```
class Question(BaseModel):
    question: str = Field(min_length=1, max_length=500)
    mode: Literal['plain'] = 'plain'
```

型と長さをサーバーで検証します。
ブラウザの `maxlength` だけでは、別のプログラムからの入力を防げません。
空白だけの文字列は `strip()` の後にも確認します。

入力検証を通る例・止まる例

question	文字数の条件	strip後の確認
"FastAPIとは？"	通る	通る
""	止まる	到達しない
" "	通る	空文字になるので止まる
501文字の文	止まる	到達しない

文字数と、空白だけかどうかは別の確認です。
入力検証で止まれば、LLM APIは呼び出しません。

エンドポイント

```
@app.post('/api/ask')
def ask(request: Question):
    question = request.question.strip()
    result = ask_plain(question)
    return result
```

これは中心部分の抜粋です。実際のコードには入力検証、
エラー処理、表示用の情報の追加があります。

FastAPIからLLMを呼ぶ

```
result = ask_plain(question)
```

第2回と同じ関数です。入力方法が `input()` から
HTTPリクエストに変わっても、LLMとの通信は再利用できます。

通常の `def` のエンドポイントから、同期SDKを呼びます。
今回は独自の非同期処理やストリーミングは追加しません。

回答のJSONに含める情報

キー	用途
answer	回答欄に表示する文字列
sources / context	第3回は空。後の回で資料を入れる
seconds	サーバーで計測した処理時間
input_tokens / output_tokens	APIが返した利用量
demo_mode	デモかAPIかの区別

回答の文字列以外も返すことで、画面で状況を確認できます。

静的ファイルの配信

```
app.mount(
    '/',
    StaticFiles(directory=BASE_DIR / 'static', html=True),
    name='static',
)
```

API定義より後に書きます。公開対象は `static/` だけです。
アプリのディレクトリ全体を公開すると、設定やDBが漏れる危険があります。

実習1：サーバー起動

リポジトリのルートから実行します。

```
source .venv/bin/activate
cd session03/exercise
python main.py
```

8000番のポートを開きます。

画面は表示されますが、送信部分が穴埋めなのでまだ質問できません。

画面表示と質問送信の違い

観察	そこまでで分かること
入力画面が開く	HTML・CSSなどを取得できた
モード表示が出る	/api/config の取得に成功した
TODOのメッセージが出る	送信処理の未実装箇所へ到達した
回答が表示される	質問送信から表示まで通った

画面が開くだけでは、質問のPOST通信は確認できません。

fetchによる送信

```
const response = await fetch('/api/ask', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({question: question, mode: 'plain'})
});
const data = await response.json();
```

相対パスなので同じサーバーへ送ります。

JavaScriptのオブジェクトを `JSON.stringify()` でJSON文字列にします。

asyncとawait

`fetch()` の通信結果は、すぐには返りません。

- `async function` ：待ち時間のある処理を書く関数
- `await` ：その処理の結果を待って次へ進む
- その間もブラウザ全体が固まるわけではない

`response.json()` も、本文の読み取りを待ちます。

待機する処理の順番

1. `await fetch(...)` でレスポンスを待つ
2. `await response.json()` で本文を読み取り、JSONを解析する
3. `response.ok` を確認する
4. 成功なら回答を表示し、失敗ならエラー表示へ進む

HTTPエラーの本文も、教材のAPIはJSONで返します。
本文の読み取りや通信自体が失敗した場合も `catch` へ進みます。

実習2：POSTの穴埋め

1. `static/app.js` の `TODO` を探す
2. POST通信のコメントを外す
3. 仮の `throw new Error(...)` を削除する
4. ページを再読み込みして送信する
5. デモの固定文が表示されることを確認する

次に講師指定のAPIモードで同じ操作を確認します。

Networkタブで観察

開発者ツールのNetworkタブで `/api/ask` を選びます。

- Request Payload：送った質問と方式
- Status：200、422、502など
- Response：返ってきたJSON
- 時間：応答までの待ち時間

外部LLMへの通信はサーバー側なので、ここには直接表示されません。

Networkタブで記録する1件

質問送信後に、一覧から `/api/ask` を選びます。

欄	確認する値
Method	POST
Request Payload	入力したquestion、modeはplainか
Status	正常なら200、不正な入力なら422など
Response	answer、demo_mode、利用量など

一覧が空なら、送信前からタブを開いて再度試します。
外部LLMへの通信は、ここに表示される1件とは別です。

待っている間の画面

```
button.disabled = true;
statusText.textContent = '調べています…';
```

処理の最後には `finally` でボタンを元に戻します。
成功時も失敗時も、次の操作ができる必要があります。

送信前に古い回答と資料を消し、新しい質問の結果と混同しないようにします。

成功と失敗の後の画面

状態	回答欄	ボタン
送信直後	古い回答を消す	無効
成功後	新しい回答を表示	有効に戻す
失敗後	古い回答を再表示しない	有効に戻す

`finally` は成功・失敗の両方で実行されます。
失敗後に入力を直して再送できるかも確認します。

HTTPエラーの扱い

`fetch()` は、HTTP 400や500でも必ず例外になるわけではありません。

```
if (!response.ok) {
  throw new Error('入力内容やAPI設定を確認してください。');
}
```

通信自体の失敗と、サーバーが返したエラーを区別します。

エラーが起きた場所

観察	最初に確認する場所
/api/ask が送られていない	app.js、Console、穴埋めの状態
/api/ask が422を返す	送信したJSONと入力の条件
/api/ask が502を返す	サーバー側のAPI設定
200だが回答が表示されない	Responseのanswerと表示処理

「答えが出ない」を、通信・処理・表示に分けて調べます。

安全な表示

```
document.getElementById('answer').textContent = data.answer;
```

LLMの出力も、ユーザー入力と同じように扱います。
`innerHTML` へ入れると、HTMLとして解釈してしまいます。

教材ではMarkdownの装飾を付けず、文字列として表示します。

実習3：異常な入力

`/docs` を開き、POST `/api/ask` を試します。

入力	期待する結果
空文字	422
空白だけ	422
501文字以上	422
`mode: "unknown"`	422

ブラウザの制約を通さなくても、サーバー側が止めます。

実習の完了確認

- デモで送信し、固定文が表示される
- Networkタブで、送った質問と200の応答を確認する
- `/docs` から不正な入力を送り、422を確認する
- 画面に戻り、正常な質問を再送できる

API環境を使える場合は、同じ画面から生成も確認します。
記録には、デモで確認した項目とAPIで確認した項目を分けます。

確認問題

1. `fetch('/api/ask')` は誰に送信している？
2. APIキーが必要なのは、どちらの通信？
3. `finally` がないと、失敗後の画面で何が起こり得る？
4. LLMが返したHTMLをそのまま実行してよい？

提出物と次回

提出：`static/app.js` と `worksheet.md` のGitHub URL。

ワークシートには、質問から回答表示までの処理を4段階で書きます。
HTTP 422を1例確認して記録します。

次回は、LLMへ授業資料を渡し、回答の根拠を確認します。